

Повний план окремої schema-aware статті

When Does Schema Context Help? A Controlled Study of Catalog-Grounded SQL Auditing for Text-to-SQL Benchmarks

Робочий документ • Volodymyr Sabadosh, Vladyslav Kotsovsky • Uzhhorod National University
Пряме продовження AST-only роботи UJIT; окрема лінія від calibration/reconciliation article

Версія 1.0 • 17 серпня 2026

Головна теза. AST показує структурну форму SQL, але не може самостійно визначити, чи існує колонка, чи є вона nullable, чи задає GROUP BY функціональну залежність, чи JOIN множить рядки та чи ORDER BY встановлює повний порядок. Нова робота має не просто додати metadata до детектора, а *контрольовано виміряти*, коли catalog grounding пригнічує false positives, знаходить нові ризики або змушує систему чесно утриматися від висновку.

Вердикт щодо публікабельності. Сильна стаття можлива як дослідження *incremental value of schema context*: той самий SQL і той самий AST-rule family порівнюються в режимах AST-only, declared schema, native catalog і consistency-checked catalog. Варіант «ми додали РК/FK до лінера» недостатній, оскільки SQLCheck уже використовував контекст, а новий інструмент sqlsure уже перевіряє join cardinality на Spider і BIRD.

Обов'язково для публікабельної версії

Публічні посилання, які мають бути в документі та майбутній статті:

- Наша опублікована UJIT-робота: [Automated Detection and Comparative Analysis of SQL Antipatterns in Text-to-SQL Datasets](#) (прямий PDF).
- Новий споріднений інструмент, який уточнив дизайн цієї наступної роботи: [sqlsure](#); аналізований frozen snapshot: [commit d1b98e7](#).
- Дослідницька система SQLCheck: [SQLCheck: Automated Detection and Diagnosis of SQL Anti-Patterns](#).

Зміст

1	Executive summary і межі роботи	4
1.1	Що саме будемо	4
1.2	Що є новим результатом	4
1.3	Що свідомо не входить	4
2	Чітка дельта до наших попередніх робіт	4
2.1	UJIT 2026 як C0 baseline	4
2.2	e-Informatica як infrastructure baseline	4
2.3	План calibration та reconciliation залишається окремим	5

3 SQLCheck і sqlsure: що вже зроблено іншими	5
3.1 SQLCheck	5
3.2 Що реально реалізує sqlsure	6
3.3 Важливі обмеження sqlsure для fair comparison	6
3.4 Хронологія і priority	6
4 Research gap, RQs і гіпотези	7
4.1 Точний gap	7
4.2 Research questions	7
4.3 Pre-specified hypotheses	7
5 Архітектура запропонованої системи	8
5.1 Data flow	8
5.2 Не дублювати materialization	8
5.3 Модель контексту	8
5.4 Provenance і reliability	9
5.5 Lineage resolver	9
5.6 RuleDecision і risk vector	9
6 Які наявні rules виграють від schema context	10
7 Primary schema-aware rules: формальні ідеї та приклади	11
7.1 R1: nullable NOT IN	11
7.2 R2: functional-dependency-aware Missing GROUP BY	11
7.3 R3: total-order-aware LIMIT/OFFSET	12
7.4 R4: type/index-aware SARGability	12
7.5 R5: schema-grounded Cartesian/join diagnosis	12
7.6 R6: key-aware Redundant DISTINCT	12
8 Три нові rules, потрібні для Text-to-SQL	12
8.1 N1: aggregate after fan-out або chasm	12
8.2 N2: join-key mismatch	13
8.3 N3: potential multi-row scalar subquery	13
8.4 Optional: outer-join null rejection	13
9 Multi-dimensional risk model	13
9.1 Чому single severity недостатня	13
10 Експериментальний протокол	14
10.1 Corpora і версії	14
10.2 Conditions	14
10.3 Development/holdout discipline	14
10.4 Baselines	15
10.5 Matched synthetic tests як causal sanity check	15
10.6 Blind human evaluation	15
10.7 Metrics	16
10.8 Performance subset	16
11 Schema reliability і query-impact graph	16
11.1 Навіщо перевіряти джерело фактів	16
11.2 Schema-to-query impact graph	17

12 Evidence records, counterexamples і відмова від auto-repair	17
12.1 Evidence bundle	17
12.2 Мінімальні witnesses	17
12.3 Чому не робимо Repair	17
13 Implementation blueprint	17
13.1 Existing integration points	17
13.2 Proposed modules	18
13.3 Backward-compatible API	18
13.4 Tests	18
13.5 Implementation phases	19
14 План майбутньої статті	19
14.1 Рекомендована назва	19
14.2 Однореченнєва теза	19
14.3 Заплановані внески	19
14.4 Abstract blueprint	20
14.5 Section outline	20
14.6 Заплановані figures і tables	20
15 Claims: що можна і чого не можна заявляти	21
15.1 Допустимі claims після успішного experiment	21
15.2 Claims, яких слід уникати	21
16 Threats to validity	21
17 Decision gates	22
18 Artifact і reproducibility checklist	22
19 Практичний порядок наступних дій	23
20 Ключові джерела та посилання	23

1. Executive summary і межі роботи

1.1. Що саме будемо

До наявного dialect-aware AST analyzer додається спільний SchemaContext, який отримується двома шляхами:

1. із native database через catalog reflection;
2. із DDL/sql_context: наявний materializer створює порожню БД, після чого використовується той самий reflection path.

Rule engine залишається чистим: він не відкриває DB connection і не виконує SQL. Він отримує AST, immutable context та повертає evidence-backed рішення. Запит не запускається вдруге; execution analyzer продовжує виконувати його один раз у своєму окремому pipeline step.

1.2. Що є новим результатом

- Контрольований C0–C3 ablation, а не повторний corpus census.
- Schema facts із provenance і reliability state.
- Scope/alias/CTE/derived-table lineage, потрібний для коректного зв'язування AST із колонками.
- Тризначна семантика present/absent/unknown замість silent-safe.
- Незалежний risk vector: correctness, performance, portability, maintainability.
- Нова blind annotation set для disagreement cases.
- Систематичне порівняння з sqlsure лише на спільних rule families.

1.3. Що свідомо не входить

- Повна NL–SQL semantic correctness evaluation: це належить окремій статті про calibration та reconciliation.
- Автоматичне переписування SQL. Дозволена лише текстова remediation hint, підкріплена evidence.
- Універсальний optimizer oracle для всіх діалектів.
- Partition-aware optimization: Spider/BIRD SQLite не надають відповідного основного експериментального середовища.
- Твердження, що відсутній FK доводить неправильність JOIN.

2. Чітка дельта до наших попередніх робіт

2.1. UJIT 2026 як C0 baseline

UJIT-стаття реалізувала 14 AST-based antipattern types, dialect-specific severity profiles та corpus-level аналіз 122 802 SQL-запитів у Spider, BIRD і Gretel. Її ключове обмеження сформульовано прямо: exclusive reliance on AST-level static analysis без column types, nullability, indexes, partitioning та execution-plan evidence. У future work названо schema-aware detection, severity calibration і multi-dimensional risk model.

2.2. e-Informatica як infrastructure baseline

e-Informatica manuscript уже має:

Вимір	UJIT	Нова schema-aware робота
Основний об'єкт	AST signatures і corpus profiles	Зміна рішення одного rule family після catalog grounding
Контекст	SQL AST + dialect profile	AST + resolved lineage + schema facts + provenance
Вердикт	Boolean flag + single severity	present/absent/unknown+ risk vector
Оцінювання	Prevalence і порівняння із C++ SqlCheck	Matched ablation, blind disagreement review, sqlsure baseline
Новизна	Corpus-scale AST antipattern audit	Measured incremental value and limits of schema context

Табл. 1: Дельта до UJIT 2026.

- schema validation analyzer;
- DbManager і adapter protocol;
- materialization DB із DDL, якщо native DB відсутня;
- column type normalization, PK/FK reflection, referential-integrity checks;
- syntax, execution, AST-antipattern та LLM-semantic analyzers.

Але schema analyzer та query antipattern analyzer працюють паралельно. Нова робота з'єднає їх: schema facts стають входом для query-level rules. Старі schema totals, execution totals, 17 Cartesian cases і 263 semantic labels не подаються як нові результати.

2.3. План calibration та reconciliation залишається окремим

Calibration article оцінює semantic defect prevalence та recall LLM committee, а також item-level reconciliation із SQLDriller. Поточна стаття оцінює structural schema-conditioned propositions. Вона не використовує старі 263 labels як ground truth і не намагається дати новий загальний «відсоток неправильних gold SQL».

Методологічне застереження

У майбутній статті потрібен окремий підрозділ *Relation to our prior work*. Без нього рецензент може сприйняти C0 і повторне використання корпусів як salami slicing. Захистом є нова одиниця аналізу: *paired rule decision under changing context*, а не повторна таблиця prevalence.

3. SQLCheck і sqlsure: що вже зроблено іншими

3.1. SQLCheck

SQLCheck SIGMOD 2020 сформулював pipeline context formation, anti-pattern detection, ranking і repair для database applications. Тому не можна заявляти «first schema-aware SQL antipattern detector». Наша дельта: gold-query benchmark auditing, ідентичний AST-only/schema-aware ablation, provenance, abstention і незалежна емпірична оцінка.

Публічно відтворюваний C++ [SqlCheck](#) є іншим artifact і вже використовувався в UJIT як baseline. Його можна залишити secondary baseline лише для overlapping syntax rules; він не є центральним schema-aware конкурентом.

3.2. Що реально реалізує sqlsure

Станом на 17 серпня 2026 року sqlsure є open-source artifact v0.1.1, а не рецензованою статтею. Його core:

- Python + SQLGlot;
- semantic model із table grain, measures, sensitive columns і declared joins;
- SQLite PRAGMA та PostgreSQL/MySQL information_schema introspection для PK/FK;
- дев'ять rules: FANOUT, CHASM, ADDITIVITY, SEMI_ADDITIVE, WEIGHTED_AVG, JOIN_KEY, UNDECLARED_JOIN, CROSS_JOIN, SENSITIVE_COLUMN.

Catalog автоматично дає grain і join cardinality. Additivity, semi-additive dimension і sensitive policy потребують semantic metadata або ручної декларації. Інструмент не читає rows, не перевіряє RI, не є data-aware й не знає NL intent.

3.3. Важливі обмеження sqlsure для fair comparison

Аналіз frozen snapshot d1b98e7 показує:

- parser scope-aware, але не повністю lineage-aware;
- CTE output lineage не переноситься в outer scopes;
- derived-table joins можуть бути пропущені;
- partial composite-key match може помилково пройти як valid join;
- ON TRUE та ON 1=1 мають predicate і не позначаються CROSS_JOIN;
- порожній список violations іноді означає «не вдалося атрибутувати», а не доведене safe;
- Spider і BIRD audit scripts викликають extraction layer та окремо класифікують join pairs із назв колонок; це не тотожне запуску всіх дев'яти check() rules;
- «zero false alarms» базується на retrospective manual review candidates і не є blind evaluation всього rule set.

Науково коректна позиція. sqlsure є корисним, пізнішим і вузьким semantic-model checker. Він сильніший за UJIT у cardinality/additivity domain, але не перекриває UJIT taxonomy або e-Informatica pipeline. Нова стаття повинна цитувати artifact, pin exact commit і використовувати його як baseline, а не знецінювати.

3.4. Хронологія і priority

- UJIT article received: 31 березня 2026.
- UJIT article accepted: 22 квітня 2026.
- sqlsure repository created: 2 липня 2026.

Отже, UJIT має чіткий хронологічний priority для опублікованого corpus-scale SQLGlot AST audit. sqlsure впливає на позиціонування наступної schema-aware статті, але не ретроспективно на новизну UJIT.

4. Research gap, RQs і гіпотези

4.1. Точний gap

У знайденій літературі та artifacts немає peer-reviewed контрольованого дослідження, яке на одному й тому самому rule engine із незалежними labels вимірює incremental value of declared, native та consistency-checked schema facts для аудиту reference SQL, включно з explicit abstention за неповного metadata.

4.2. Research questions

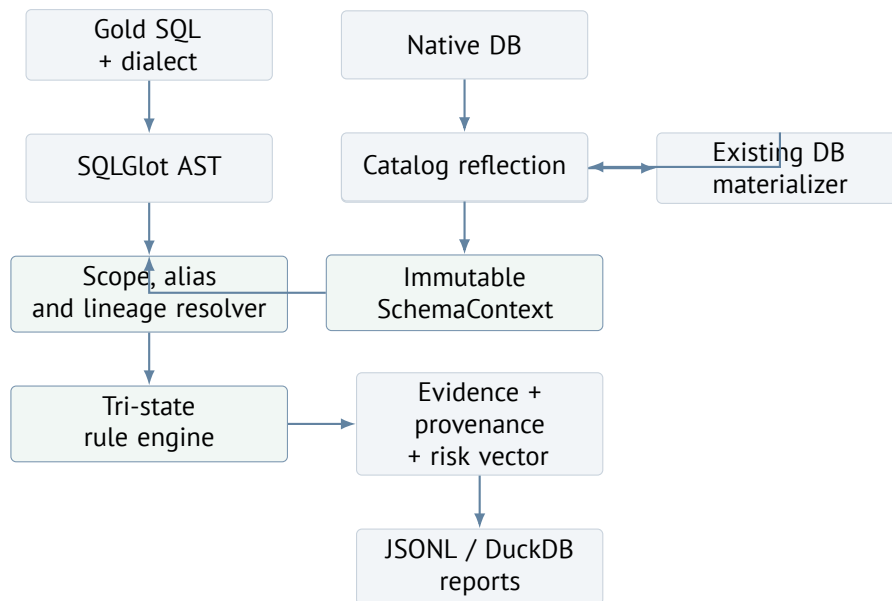
- RQ1** Наскільки повними, узгодженими й надійними є names, types, nullability, PK/FK, uniqueness та indexes у Spider, BIRD і Gretel?
- RQ2** Які AST-only findings context підтверджує, пригнічує, додає або переводить у unknown?
- RQ3** Як context змінює precision, false-positive rate, review yield і risk-vector assignments?
- RQ4** Які rule families найбільше виграють від nullability, key, cardinality, type, index і lineage facts?
- RQ5** Які підтверджені cases унікальні для нашого analyzer, sqlsure, DBMS resolution та execution check?
- RQ6** Як acquisition path — native database проти DDL-materialized schema — впливає на coverage, abstention і throughput?

4.3. Pre-specified hypotheses

- **H1:** context-dependent rules матимуть вищу precision у C1–C3, ніж у C0.
- **H2:** syntax-only negative-control rules не зміняться між conditions.
- **H3:** найбільше false-positive suppression дасть nullable NOT IN.
- **H4:** найбільше нових correctness-risk findings дадуть join-key mismatch, fan-out/chasm і potential multi-row scalar subquery.
- **H5:** Gretel матиме більше unknown через неповний або неоднорідний DDL, але materialization дозволить застосувати declaration-based rules масштабно.
- **H6:** published metadata, native catalog і consistency checks на даних дадуть не тотожні rule decisions.
- **H7:** lineage-aware аналіз матиме вищу applicable coverage на CTE та derived-table queries, ніж sqlsure.

5. Архітектура запропонованої системи

5.1. Data flow



5.2. Не дублювати materialization

Наявний lifecycle уже створює database/schema з DDL за відсутності native DB. Нова реалізація не переносить цей код у detector. На analyzer boundary:

1. normalizer отримує schema/DDDL;
2. adapter canonicalizes DDL і формує stable schema identity;
3. DB створюється лише за потреби;
4. DbManager повертає tables і table info;
5. SchemaContextBuilder freezes facts і кешує context за db_id або canonical schema hash;
6. pure detector використовує context без I/O.

Для Gretel однакові DDL мають дедуплікуватися за canonical hash. Порожня materialized DB дозволяє names/types/nullability/PK/FK/unique/index rules, але не data-derived claims.

5.3. Модель контексту

```

@dataclass(frozen=True)
class ColumnFact:
    name: str
    type_raw: str
    type_family: str
    nullable: bool | None
    primary_key: bool
    unique: bool | None
    provenance: FactProvenance

@dataclass(frozen=True)
class TableFact:
    name: str
    columns: Mapping[str, ColumnFact]
    primary_key: tuple[str, ...]
  
```



```
unique_keys: tuple[tuple[str, ...], ...]
foreign_keys: tuple[ForeignKeyFact, ...]
indexes: tuple[IndexFact, ...]
```

```
@dataclass(frozen=True)
```

```
class SchemaContext:
```

```
    db_id: str
```

```
    dialect: str
```

```
    source: str
```

```
    tables: Mapping[str, TableFact]
```

5.4. Provenance i reliability

Provenance	Що означає	Дозволені висновки
benchmark_metadata	Факт узято з tables.json або іншого benchmark manifest	Declaration-based; missing fact не є доказом відсутності
materialized_from_ddl	Catalog створено з DDL без population data	Names, types, declared keys, nullability, indexes
native_catalog	Факт прочитано з shipped DB catalog	Catalog-level reasoning; ще не доказ data consistency
data_consistency_checked	Declared constraint не суперечить поточному instance	Підвищена довіра лише для цього snapshot; не універсальний доказ
observed_only	Uniqueness/null absence спостережено в rows, але не declared	Supporting evidence; не підмінює logical constraint

Табл. 2: Джерела schema facts та межі інтерпретації.

5.5. Lineage resolver

Resolver повинен:

- будувати lexical score для кожного SELECT;
- відображати alias у base table;
- вирішувати qualified та unqualified columns;
- виявляти ambiguity, а не вгадувати owner;
- переносити output-column lineage через CTE та derived tables;
- підтримувати projection aliases, USING, composite keys та nested scopes;
- позначати unsupported expression lineage як unknown.

Це важливіше за додавання десятків rules: помилкова атрибуція колонки робить будь-яке schema-aware правило недостовірним.

5.6. RuleDecision i risk vector

```
@dataclass(frozen=True)
```

```
class RuleDecision:
```

```
    rule_id: str
```

```
    state: Literal["present", "absent", "unknown"]
```

```
evidence: tuple[Evidence, ...]
provenance: tuple[FactProvenance, ...]
correctness: RiskLevel | None
performance: RiskLevel | None
portability: RiskLevel | None
maintainability: RiskLevel | None
remediation_hint: str | None
```

Legacy Critical/High/Medium/Low залишається лише для backward compatibility. У науковому аналізі не слід зводити correctness і performance до одного числа без емпірично валідованих ваг.

6. Які наявні rules виграють від schema context

Наявний rule	Виграш	Schema-aware refinement
NOT IN nullable	Дуже високий	Визначити nullability projected expression, включно з outer-join/CASE lineage; не flag усі NOT IN автоматично.
Missing GROUP BY	Високий	Перевірити PK, composite/UNIQUE NOT NULL keys та functional dependencies; розділити correctness і portability.
LIMIT/OFFSET ordering	Високий	Не лише наявність ORDER BY, а total order відносно result grain.
Function in WHERE	Високий за наявності metadata	Типи, implicit conversion, ordinary/expression indexes; EXPLAIN subset validation.
Cartesian product	Середній	Краща alias ownership, FK path evidence, wrong-table/self-reference diagnosis; сам Cartesian факт переважно AST-based.
Redundant DISTINCT	Середній–високий	PK/unique/FD дозволяють довести, що output уже унікальний.
SELECT *	Середній для severity	Розгорнути projection; BLOB/JSON/wide columns підвищують resource risk; sensitive metadata може додати policy risk.
Leading wildcard LIKE	Середній для performance	Index/collation/type metadata; без індексу не стверджувати «index disabled».
Correlated subquery	Середній	Lineage підтверджує реальну correlation; index metadata уточнює performance risk.
NULL comparison	Мінімальний	= NULL є некоректною predicate form незалежно від schema.
Unsafe UPDATE/DELETE	Мінімальний	Відсутність WHERE визначається AST; schema не змінює основний verdict.
Columns in EXISTS	Мінімальний	Projection усередині EXISTS семантично ігнорується; AST достатньо.

Табл. 3: Вплив schema context на опублікований UJIT rule set.

Обов'язково для публікабельної версії

Primary article scope: поглибити шість наявних families — nullable NOT IN, Missing GROUP BY, LIMIT/OFFSET total order, Function in WHERE, schema-grounded Cartesian/join diagnosis і key-aware Redundant DISTINCT. SELECT *, leading wildcard і correlated subquery залишити secondary analyses.

7. Primary schema-aware rules: формальні ідеї та приклади**7.1. R1: nullable NOT IN**

Required facts: lineage projected subquery expression, declared nullability, null-introducing operators.

- present: expression може бути NULL за declared schema або expression semantics.
- absent: expression доведено NOT NULL у цьому scope.
- unknown: unresolved lineage або missing nullability fact.

```
SELECT u.id
FROM users AS u
WHERE u.id NOT IN (
  SELECT b.user_id
  FROM blocked_users AS b
);
```

Якщо `blocked_users.user_id` nullable, один NULL може зробити predicate UNKNOWN для всіх candidate rows. Якщо колонка NOT NULL, AST-only flag пригнічується. Якщо projection є CASE або походить із nullable side outer join, простого column-level NOT NULL недостатньо.

7.2. R2: functional-dependency-aware Missing GROUP BY

```
CREATE TABLE customers (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL
);

SELECT c.id, c.name, COUNT(o.id)
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id
GROUP BY c.id;
```

AST-only detector бачить `c.name` поза GROUP BY. Schema-aware detector доводить:

$$\text{customers.id} \rightarrow \text{customers.name.}$$

Correctness risk стає absent; portability може залишитися dialect-dependent.

Контрприклад:

```
SELECT country, city, COUNT(*)
FROM customers
GROUP BY country;
```

Якщо `country` не покриває PK або UNIQUE NOT NULL key, `city` не є функціонально залежною і SQLite може вибрати довільне значення. Для nullable UNIQUE потрібна обережність: кілька NULL можуть утворити одну групу. Для composite key потрібні всі key columns.

7.3. R3: total-order-aware LIMIT/OFFSET

```
SELECT id, score
FROM candidates
ORDER BY score DESC
LIMIT 10;
```

Наявність ORDER BY ще не гарантує deterministic top-10: ties за score дозволяють різний subset. Якщо score не unique, recommendation: ORDER BY score DESC, id. Rule має враховувати result grain після GROUP BY, DISTINCT і joins; якщо grain не виведено, verdict unknown.

7.4. R4: type/index-aware SARGability

```
SELECT *
FROM events
WHERE CAST(user_id AS TEXT) = '42';
```

AST-only rule бачить function/cast. Schema-aware version перевіряє:

- actual type family user_id;
- implicit/explicit conversion direction;
- ordinary або expression index;
- dialect-specific coercion;
- plan evidence на підтримуваній subset через EXPLAIN QUERY PLAN.

Без індексу не слід стверджувати, що function «заблокувала index». Тоді може залишитися computation/portability risk, але не підтверджений index-access defect.

7.5. R5: schema-grounded Cartesian/join diagnosis

AST уже знаходить comma joins, ON TRUE, one-sided predicates та missing ON. Schema додає:

- точну ownership колонок;
- expected FK path як evidence;
- distinction між wrong key, absent predicate та unknown undeclared relation;
- diagnosis wrong-table references.

Відсутній FK не перетворюється на error: це unknown або unmodeled warning.

7.6. R6: key-aware Redundant DISTINCT

DISTINCT можна довести redundant, якщо projected columns містять result key і joins не розмножують цей grain. Навпаки, наявність PK базової таблиці не достатня після one-to-many join. Потрібні lineage і cardinality propagation.

8. Три нові rules, потрібні для Text-to-SQL

8.1. N1: aggregate after fan-out або chasm

```
SELECT c.id, SUM(c.credit_limit)
FROM customers AS c
JOIN orders AS o ON o.customer_id = c.id
GROUP BY c.id;
```

Якщо `customers.id` → `orders` є one-to-many, base-row `credit_limit` повторюється для кожного `order`. Schema facts доводять row multiplication; intent щодо measure усе ще може вимагати review.

- **FANOUT:** aggregate над measure/column із grain, який повторюється join.
- **CHASM:** дві або більше fan-out branches множать одна одну.
- **Не робити:** вважати `SUM(DISTINCT amount)` універсально безпечним, бо різні rows можуть мати однаковий amount.

Цей rule напряду порівнюється із sqlsure FANOUT/CHASM, але наш експеримент додає lineage, provenance, abstention і independent labels.

8.2. N2: join-key mismatch

```
SELECT *
FROM orders AS o
JOIN customers AS c
ON o.sales_rep_id = c.customer_id;
```

Якщо schema декларує `orders.customer_id` → `customers.customer_id`, використаний key суперечить declared relationship. Critical correctness risk допустимий лише за позитивного evidence. Якщо relationship не declared, система повертає unknown, а не автоматичний error.

Composite FK вважається matched тільки тоді, коли покрито весь key; один правильний pair не повинен маскувати другий неправильний predicate.

8.3. N3: potential multi-row scalar subquery

```
SELECT p.name,
       (SELECT r.rating
        FROM reviews AS r
        WHERE r.product_id = p.id) AS rating
FROM products AS p;
```

Якщо `reviews.product_id` не unique і немає aggregate/LIMIT із детермінованим order, scalar subquery може повернути кілька rows. PostgreSQL завершиться error; SQLite може використати один рядок, створюючи portability та determinism risk.

8.4. Optional: outer-join null rejection

```
SELECT c.id
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id
WHERE o.status = 'paid';
```

WHERE predicate відкидає NULL-extended rows і фактично робить LEFT JOIN подібним до INNER JOIN. Це важливий Text-to-SQL risk, але intent-dependent; додавати в primary study лише після pilot.

9. Multi-dimensional risk model

9.1. Чому single severity недостатня

UJIT справедливо використовував Critical/High/Medium/Low як operational prioritization. Schema context дозволяє розділити незалежні наслідки:

Вісь	Питання	Приклади
Correctness	Чи може query повернути логічно неправильні rows/value?	nullable NOT IN, fan-out SUM, wrong join key
Performance	Чи є evidence про scan, row explosion або expensive projection?	indexed function predicate, wide SELECT *, correlated execution
Portability	Чи залежить поведінка від dialect/coercion?	bare GROUP BY column, multi-row scalar subquery, type coercion
Maintainability	Чи підвищує форма query ризик майбутньої помилки?	SELECT *, undeclared join path, ambiguous ownership

Табл. 4: Незалежні risk dimensions.

Для кожної осі зберігати evidence-backed level або unknown. Не публікувати композитний quality score, доки ваги не валідовані людьми або downstream outcome.

10. Експериментальний протокол

10.1. Corpora і версії

- **Spider 1.0**: primary database-backed corpus, усі доступні partitions; exact source checksum у manifest.
- **BIRD**: відтворити dev_20240627, який використовував sqlsure, і окремо провести main evaluation на актуальному corrected snapshot, pinned до дати запуску.
- **Gretel synthetic Text-to-SQL**: DDL-derived external-validity та scalability corpus; snapshot/commit, DDL hash і deduplication обов'язкові.

Spider і BIRD є primary study. Gretel звітується окремо; його synthetic SQL не називається human gold SQL, а результати не pool-яться з database-backed corpora.

10.2. Conditions

- C0** AST-only UJIT-compatible mode.
- C1** AST + declared schema facts: benchmark metadata або DDL; за потреби acquisition через materialized empty DB.
- C2** AST + native live catalog shipped database.
- C3** AST + catalog constraints, consistency-checked against populated instance.

C3 не «доводить» general constraint: він лише показує, що current snapshot не суперечить declaration. Для Gretel primary comparison C0/C1; reflection із materialized DB є acquisition path для C1, а не незалежним source condition.

10.3. Development/holdout discipline

1. Розробляти rule logic на synthetic matched fixtures і Spider Train.
2. Freeze rule specification, severities/risk mapping і code commit.
3. Evaluate на Spider Dev/Test, corrected BIRD Dev і Gretel snapshot.
4. Known corrected BIRD/sqlsure cases використовувати лише як external validation після freeze або чесно позначити retrospective subset.

Це зменшує corpus-guided tuning, яке послаблює як UJIT regression construction, так і sqlsure retrospective claims.

10.4. Baselines

1. **C0 current analyzer**: exact published rule behavior.
2. **sqlsure v0.1.1, commit d1b98e7**: запуск public check() API.
3. **sqlsure audit scripts**: окремий результат, не змішувати з checker.
4. **DBMS prepare/resolution**: identifier/type hard errors.
5. **C++ SqlCheck v1.3**: secondary baseline для відтворюваних overlapping syntax rules.

Методологічне застереження

Не рахувати один глобальний F1 між systems із різними taxonomies. Порівняння sqlsure обмежити CROSS_JOIN, JOIN_KEY, FANOUT, CHASM та clearly mapped propositions. Additivity і sensitive policy не можна оцінювати без однакового semantic metadata.

10.5. Matched synthetic tests як causal sanity check

SQL залишається ідентичним, змінюється один schema fact:

- nullable $\leftrightarrow$ NOT NULL;
- unique $\leftrightarrow$ non-unique;
- FK present $\leftrightarrow$ absent;
- one-to-one $\leftrightarrow$ one-to-many;
- type-compatible $\leftrightarrow$ incompatible;
- index absent $\leftrightarrow$ ordinary/expression index.

Syntax-only control rules повинні залишатися інваріантними. Якщо вони змінюються, context integration має leakage або implementation bug.

10.6. Blind human evaluation

Нова annotation set формується зі страт:

- C0 present $\rightarrow$ C1/C2 absent;
- C0 absent $\rightarrow$ C1/C2 present;
- C1 $\neq$ C2 або C2 $\neq$ C3;
- proposed-only та sqlsure-only disagreements;
- random unflagged controls;
- representative unknowncases.

Два незалежні SQL/DB reviewers, blind до detector і condition, потім adjudication. Label schema:

1. підтверджена query-level structural проблема;
2. valid query, але evidence-backed portability/performance risk;
3. schema/metadata defect;
4. intentional/acceptable construction;
5. unverifiable with available evidence.

Початковий pilot визначає prevalence positives у strata; фінальний sample size обирається через power/precision calculation. Реалістичний орієнтир для focused rule-family claims — 400–600 reviewed cases, але рідкісні classes можуть потребувати census usix findings.

10.7. Metrics

- context acquisition coverage і parse/resolve coverage;
- rule applicability та abstention rate;
- retained, suppressed, added і unknown migrations;
- finding-level і query-level precision/recall/F1;
- false-positive suppression і incremental true positives;
- risk-vector migration;
- overlap/Jaccard із baselines на matched rules;
- inter-annotator agreement (κ або Krippendorff's α);
- review time, throughput, schema-load latency, memory;
- per-database clustering і concentration of findings.

Paired binary outcomes: McNemar. Confidence intervals: clustered bootstrap за db_id. Для stratified manual sample: sampling weights. Не трактувати queries з однієї DB як незалежні observations.

10.8. Performance subset

Для Function-in-WHERE, leading wildcard і SELECT-*:

1. зібрати reliable index/type facts;
2. запустити EXPLAIN QUERY PLAN на підтримуваних SQLite cases;
3. відділити logical finding від plan-confirmed evidence;
4. не переносити SQLite plan conclusion на PostgreSQL без окремого experiment.

11. Schema reliability і query-impact graph

11.1. Навіщо перевіряти джерело фактів

Published benchmark metadata, materialized DDL і native catalog можуть розходитись. Пропонований audit повинен звітувати:

- missing/extra tables і columns;
- nullability disagreements;
- PK/FK/unique/index disagreements;
- declared constraints, порушені current data;
- unsupported або failed DDL materialization;
- частку query references, які вдалося повністю resolve.

11.2. Schema-to-query impact graph

Нова корисна одиниця:

schema fact or defect $\longrightarrow$ {dependent gold queries and rule decisions}.

Наприклад, один missing FK може породити десятки unbacked-join candidates. Замість «30 помилок SQL» треба показати «один schema defect, що впливає на 30 queries». Це запобігає double counting і напряду з'єднує e-Informatica schema analyzer із query-level article.

12. Evidence records, counterexamples і відмова від auto-repair

12.1. Evidence bundle

Кожний finding повинен містити:

- rule id і precise AST location;
- resolved table/column lineage;
- schema facts і provenance;
- reason for present/absent/unknown;
- risk vector;
- short human-readable remediation hint;
- analyzer/rule/schema versions.

12.2. Мінімальні witnesses

Для кількох correctness rules можна створити невеликі counterexample instances:

- один NULL для NOT IN;
- два child rows для fan-out;
- ties для non-total ORDER BY;
- два subquery rows для scalar-subquery risk.

Witness доводить, що risk може матеріалізуватися, але не доводить, що gold SQL не відповідає конкретному NL question.

12.3. Чому не робимо Repair

Автоматичні rewrites для NOT IN, fan-out, GROUP BY або ORDER BY можуть змінити intent. У primary scope:

- detector не змінює SQL;
- повертає remediation hint;
- safe rewrite на кшталт = NULL $\rightarrow$ IS NULL залишається future extension;
- semantic equivalence checking і repair evaluation потребують окремого study.

13. Implementation blueprint

13.1. Existing integration points

- src/text2sql_pipeline/analyzers/query_antipattern/antipattern_detector.py

- .../query_antipattern/query_antipattern_analyzer.py
- .../query_antipattern/antipattern_registry.py
- .../query_antipattern/metrics.py
- src/text2sql_pipeline/db/manager.py
- src/text2sql_pipeline/db/adapters/base/protocol.py
- .../schema_validation/schema_validation_analyzer.py
- src/text2sql_pipeline/output/sinks/duckdb.py

13.2. Proposed modules

- schema_context.py: frozen facts, provenance, reliability.
- schema_context_builder.py: native/DDL materialized acquisition i cache.
- scope_resolver.py: alias, CTE, derived lineage.
- rule_decision.py: tri-state evidence та risk vector.
- schema_aware_rules.py: primary schema-dependent families.
- explain_evidence.py: optional supported-plan validation.

13.3. Backward-compatible API

```
def detect_antipatterns(
    sql: str,
    dialect: str = "sqlite",
    config=None,
    penalties=None,
    schema_context: SchemaContext | None = None,
) -> QueryAntipatternFeatures:
    ...
```

Якщо context відсутній, CO behavior зберігається. Schema-required rules не повинні мовчки робити AST guess: залежно від pre-specified policy вони повертають unknown або не-applicable decision, який окремо рахується.

13.4. Tests

- усі наявні AST-only regression tests;
- matched schema fixtures;
- native DB vs DDL-materialized acquisition equivalence;
- cache by db_id/schema hash;
- aliases, ambiguity, CTE shadowing, derived tables;
- composite PK/FK/unique keys;
- outer joins i null introduction;
- SQLite affinity та quoted identifiers;
- evidence serialization i DuckDB migrations;
- no-row database не є evidence про data null/uniqueness.

13.5. Implementation phases

1. **P0 — Specification freeze:** rule predicates, states, risk dimensions, datasets, baselines, pilot protocol.
2. **P1 — Context foundation:** immutable facts, provenance, index reflection, shared cache, both acquisition paths.
3. **P2 — Resolver:** scopes, aliases, CTE/derived lineage.
4. **P3 — Existing-rule upgrades:** NOT IN та GROUP BY first; then ordering, SARGability, joins, DISTINCT.
5. **P4 — New rules:** fan-out/chasm, join-key mismatch, scalar subquery.
6. **P5 — Outputs/tests:** tri-state evidence, risk vector, DuckDB/reporting.
7. **P6 — Pilot:** Spider Train development + held-out pilot.
8. **P7 — Full study:** C0–C3, baselines, annotation.
9. **P8 — Analysis/article/artifact:** statistics, frozen outputs, paper.

14. План майбутньої статті

14.1. Рекомендована назва

When Does Schema Context Help? A Controlled Study of Catalog-Grounded SQL Auditing for Text-to-SQL Benchmarks

Альтернативи:

- Beyond the AST: Catalog-Grounded Detection of Structural SQL Risks in Text-to-SQL Benchmarks
- From Parse Trees to Catalog Evidence: Measuring the Value of Schema Context in Gold-SQL Auditing
- Schema-Grounded Static Analysis of Text-to-SQL Reference Queries: Precision, Abstention, and Cross-Dataset Evidence

14.2. Однореченська теза

Catalog grounding materially changes which structural SQL findings are justified, but its value depends on metadata completeness, provenance, and query lineage; a reliable auditor must therefore measure both improved precision and principled abstention.

14.3. Заплановані внески

1. Context model із provenance/reliability і dual native/DDL acquisition.
2. Lineage-aware tri-state schema-conditioned rule engine.
3. Six refined UJIT families та три нові Text-to-SQL-relevant rules.
4. Controlled C0–C3 study на Spider/BIRD із Gretel DDL stress test.
5. Blind disagreement labels і fair comparison із sqlsure.
6. Reproducible artifact із evidence, manifests, rules і analysis scripts.

14.4. Abstract blueprint

1. **Problem:** executable gold SQL може містити risks, яких AST-only або execution-only checks не розрізняють.
2. **Gap:** schema-aware systems існують, але incremental value, metadata reliability і abstention систематично не виміряні для reference-query audits.
3. **Method:** lineage-aware context, tri-state rules, risk vector, C0–C3.
4. **Study:** Spider/BIRD primary, Gretel DDL external validity, sqlsure baseline, blind human labels.
5. **Results:** вставити лише після freeze та evaluation: suppression, additions, unknown rates, precision, coverage, runtime.
6. **Implication:** benchmark QA має поєднувати AST, trustworthy catalog facts, execution і semantic review, не змішуючи їхні claims.

14.5. Section outline

1. Introduction and contributions.
2. Background and relation to prior work.
3. Catalog-grounded analysis model.
4. Schema acquisition, provenance and lineage.
5. Rule specifications and risk model.
6. Study design.
7. Metadata reliability results.
8. C0–C3 ablation and human accuracy.
9. Comparison with sqlsure and other baselines.
10. Cross-dataset results and Gretel stress test.
11. Discussion, threats and implications.
12. Conclusion and artifact.

14.6. Заплановані figures і tables

- Figure 1: AST + native/DDL catalog architecture.
- Figure 2: scope/lineage propagation example.
- Figure 3: decision flow present/absent/unknown.
- Table 1: explicit delta from UJIT/e-Informatica/calibration.
- Table 2: rule specifications та required facts.
- Table 3: metadata coverage/reliability by corpus.
- Table 4: C0–C3 migration matrix.
- Table 5: blind precision/recall by rule family.
- Table 6: matched overlap with sqlsure.
- Table 7: runtime, cache and materialization cost.
- Figure 4: schema-defect to affected-query graph.

15. Claims: що можна і чого не можна заявляти

15.1. Допустимі claims після успішного experiment

- schema context statistically significantly improves precision for specified rule families;
- explicit abstention prevents unsupported conclusions under incomplete metadata;
- provenance sources lead to measurably different rule decisions;
- lineage increases applicable coverage on nested SQL;
- proposed system complements sqlsure on non-overlapping nullability, FD, ordering, type та scalar-subquery risks;
- DDL materialization enables scalable schema-conditioned auditing without populated databases, with clearly stated limits.

15.2. Claims, яких слід уникати

Ризик для claims

- «First schema-aware SQL antipattern detector».
- «First schema-aware audit of Spider/BIRD gold SQL».
- «Every flag is a wrong gold query».
- «Missing FK proves a wrong join».
- «Observed uniqueness proves a key».
- «Execution success validates semantics».
- «Gretel має 100 000 повноцінних populated databases».
- «sqlsure has zero false positives» без відтворення його exact protocol.
- «Наш method dominates sqlsure» за глобальним score для різних taxonomies.
- «Schema awareness solves NL–SQL intent mismatch».

16. Threats to validity

Threat	Mitigation
Missing constraints	Absence → unknown; provenance і coverage reporting.
Incorrect declarations	Compare metadata/native catalog; consistency checks; schema-defect label.
SQLite permissive typing	Coarse type families; portability axis; no universal correctness claim.
Current data cannot prove constraints	Separate declared, data-consistent і observed-only facts.
Intent unknown	Label structural proposition, not whole NL–SQL correctness.
Correlated queries per DB	Clustered inference by db_id.

Threat	Mitigation
Rule tuning on known cases	Synthetic/Spider Train development; freeze; held-out BIRD/Spider evaluation.
Gretel duplication and heterogeneity	DDL and canonical SQL hashes; separate reporting.
Baseline taxonomy mismatch	Matched rules only; pin versions and parser coverage.
Reviewer bias	Blind dual review and adjudication.
Publication overlap	Explicit relation-to-prior-work section; no recycled headline results.
Rapidly changing related work	Repeat literature and sqlsure status search immediately before submission.

Табл. 5: Основні threats і mitigations.

17. Decision gates

- G1** Metadata coverage достатнє для заявлених rules. Інакше primary scope звужується до Spider/BIRD.
- G2** Pilot NOT IN + join/cardinality показує precision/abstention improvement, а не лише інші counts.
- G3** Resolver проходить nested/alias/composite-key fixture suite без silent guesses.
- G4** Є достатньо позитивів для human evaluation кожної headline family; рідкісні classes подаються descriptive.
- G5** sqlsure comparison відтворюваний на pinned commit і однакових inputs.
- G6** Abstract не містить старих UJIT/e-Informatica чисел як нових results.
- G7** Перед submission перевірено, чи не з'явилась sqlsure paper/preprint.

Методологічне застереження

Якщо G2 не проходить, це не автоматично провал. Можлива чесна negative-result study або tool paper про те, що incomplete schema metadata дає менший виграш, ніж очікувалося. Але не слід наперед писати сильний abstract без pilot evidence.

18. Artifact і reproducibility checklist

- pinned code commit;
- dataset/version manifest;
- checksums;
- DDL normalization version;
- schema-context serialization;
- rule specifications;
- matched fixtures;
- baseline wrappers;
- sqlsure pinned snapshot;
- annotation guideline;

- blinded labels;
- adjudication log;
- statistical scripts;
- raw migration matrices;
- runtime environment;
- generated reports;
- known limitations;
- relation-to-prior-work statement.

19. Практичний порядок наступних дій

1. Визначити canonical code repository/branch і freeze C0 behavior.
2. Зробити schema feasibility inventory для Spider, BIRD, Gretel.
3. Реалізувати context builder з native та DDL-materialized paths.
4. Реалізувати resolver до додавання rules.
5. Зробити matched pilot для NOT IN і join/cardinality.
6. Прогнати decision gates G1–G3.
7. Додати решту primary rules і outputs.
8. Freeze specifications і evaluation commit.
9. Запустити C0–C3 та pinned baselines.
10. Сформувати blind disagreement sample.
11. Провести review/adjudication і statistics.
12. Написати Results лише після завершення аналізу.
13. Випустити artifact разом із paper/preprint.

20. Ключові джерела та посилання

1. Sabadosh, V. M., Kotsovsky, V. M. (2026). *Automated Detection and Comparative Analysis of SQL Antipatterns in Text-to-SQL Datasets*. Ukrainian Journal of Information Technology, 8(1), 135–143. PDF.
2. Dintyala, P., Narechania, A., Arulraj, J. (2020). *SQLCheck: Automated Detection and Diagnosis of SQL Anti-Patterns*. ACM SIGMOD.
3. Arora, T. (2026). *sqlsure: Semantic inspector for SQL*, software artifact v0.1.1. Frozen snapshot used in this plan.
4. Yu, T., et al. (2018). *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task*.
5. Li, J., et al. (2023). *Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs*.
6. Liu, X., et al. (2025). *NL2SQL-BUGs: A Benchmark for Detecting Semantic Errors in NL2SQL Translation*.
7. SQLGlot contributors. *SQLGlot: Python SQL Parser and Transpiler*.
8. Gretel.ai. *synthetic_text_to_sql dataset*.

Окремий planning artifact. Не замінює наявний FullArticlePlan для calibration/reconciliation study.